# Lab: Favorites List — UITableView & Custom Cells

Oct 29, 2026 · @Arman Myrzakanurov

## Overview

Build a UIKit app that shows a list of your favorite things in a `UITableView` with a custom cell: image, title, subtitle and a rating drawn with SF Symbols. You choose the theme. Time: **1 week**.

By the end of this lab you should be able to:

1. Configure a `UITableView` and a prototype cell in Interface Builder.
2. Lay out a custom cell with Auto Layout inside the table view in `Main.storyboard`.
3. Create a `UITableViewCell` subclass and connect its `@IBOutlet`s.
4. Dequeue reusable cells by reuse identifier in `UITableViewDataSource`.
5. Model list data with a struct and fill cells from it.

## Requirements

### 1. Theme and data

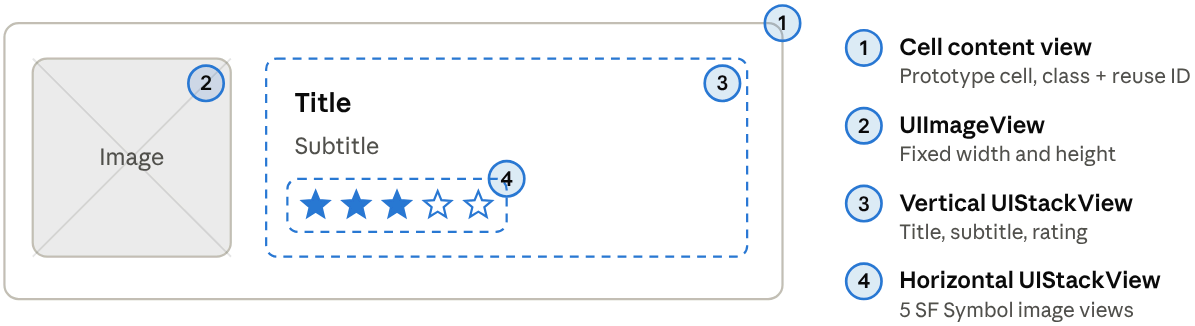
- Choose one theme for your list: Food, Movies, Countries, Books, Games, Cars, etc. Two students in the same group should not pick an identical list.
- The list contains **at least 20 items**.
- Each item has a title, a subtitle, an image and a rating from 0 to 5.
- Describe an item with a struct (e.g. `FavoriteItem`) and keep the array of items in one place, not inside the cell.
- Images are added to `Assets.xcassets`; every item has its own image.

### 2. Custom cell

- Create a separate `UITableViewCell` subclass (e.g. `FavoriteCell`) in its own Swift file.

- The class contains only `@IBOutlet`s connected from the storyboard, plus an optional method that fills the cell from one item. No layout code.
- The cell shows:
  - **Image** — `UIImageView` with a fixed size.
  - **Title** — `UILabel`.
  - **Subtitle** — `UILabel`.
  - **Rating** — a row of 5 small SF Symbol images (`star.fill` / `star`, `heart.fill` / `heart`, or another symbol that fits your theme); filled symbols show the item's rating.

**One prototype cell: image left, text and rating stacked right**




Custom cell layout · suggested structure

A suggested structure, not a pixel spec: dashed boxes are stack views, so constraints are only needed on the image and on the outer stack.

## 3. Table view

- One screen with a `UITableView` that fills the Safe Area.
- The cell is designed as a **prototype cell inside the table view** in `Main.storyboard`, with its class set to your custom cell and a **reuse identifier** set in the Attributes inspector.
- The view controller implements `UITableViewDataSource` and returns one row per item.
- Cells are created with `dequeueReusableCell(withIdentifier:for:)` using that identifier and cast to your custom class.

### 4. Layout (Main.storyboard only)

- All Auto Layout is done in `Main.storyboard`, inside the prototype cell's content view. **No layout code**: no constraints, anchors or frames in Swift.
- Image on the left; title, subtitle and rating stacked to its right.
- Long titles do not overlap the image or go off screen.
- Zero Auto Layout warnings or errors in Interface Builder.
- The list looks correct on **iPhone SE (3rd gen)** and **iPhone 16 Pro Max**, in portrait.

## Notes

- A table view does not create 20 cells for 20 items. It creates only enough to fill the screen and reuses them while you scroll. That is why every cell must be fully reset from its item each time it is dequeued.
- The reuse identifier in the storyboard and the one in your code must match exactly, or the app crashes on launch.
- Setting the custom class on the prototype cell is what lets you connect outlets to your cell class instead of the view controller.
- For the cell's height to follow its content, the constraints inside the cell must connect from the top edge to the bottom edge without gaps.
- Stack views will save you most of the constraints inside the cell.

## Deliverables

Submit before your next lab:

- [ ] GitHub repository link (public, or shared with the instructor) with a clean `.gitignore` for Xcode.
- [ ] Two screenshots in the README: iPhone SE and iPhone 16 Pro Max.
- [ ] A short screen recording scrolling through the whole list.
- [ ] The project builds and runs with no extra setup.